ATTENDEASE · SOFTWARE ENGINEERING MINI PROJECT

Code Coverage Report

Document ID	AE-CCR-01
Version	1.0
Date	10 November 2026
Build under test	`main` @ `92bcd23` (merge of PR #113, 1 Nov 2026)
Prepared by	Arundhati K (PES1UG24CS083)
Team	Arundhati K, Anshdeep Singh Sachdeva, Atharva Ashish Vyas, Bhoomika Dayanand Jituri

Statement coverage of the AttendEase backend, measured with coverage.py.

1. Summary

93.9%	542/577	96.5%	70%
SERVICE-LAYER STATEMENTS	STATEMENTS EXECUTED	ALL APPLICATION CODE	TARGET (NFR-006)

Statement coverage of the service-layer business logic is **93.9%** (542 of 577 statements), against the NFR-006 target of 70%. Across all application code, including views, serializers, models and permissions, it is **96.5%** (1400 of 1451). Every service file is above 85%.

Verdict (TC-12, NFR-006): PASS Coverage exceeds the 70% target by 23.9 percentage points.

> CI reports a single combined figure of **94%** for the service layer. coverage.py calculates it as (statements + branches covered) ÷ (statements + branches) = (542 + 155) ÷ (577 + 168) = 93.6%. This report gives statement coverage on its own. Branch coverage is in the Branch Coverage Report (AE-BCR-01).

2. Method

Item	Detail
Operating system	Linux x86-64 (2 vCPU)
Python	3.13.17 (local run) · 3.11 in GitHub Actions CI
Framework	Django 4.2.30, Django REST Framework 3.17.2, SimpleJWT 5.5.1
Database	SQLite (Django test database, created and destroyed per run)
Test runner	Django `manage.py test` (unittest) with unittest-xml-reporting for per-test results
Coverage tool	coverage.py 7.17.0, branch measurement enabled
Static checks	Ruff 0.17.1: all checks passed · `manage.py check`: no issues
Measurement run	`coverage run --branch --source=users,academic,attendance,corrections,analytics manage.py test` (320 tests, all passed)

Service layer (NFR-006 scope)	The 7 files enforced by CI: `users/services.py`, `users/security.py`, `academic/services.py`, `attendance/services.py`, `corrections/services.py`, `corrections/notifications.py`, `analytics/services.py`
Application code	All source in the five apps, excluding tests, migrations and empty `__init__.py` files
CI gate	`coverage report --include=<service files> --fail-under=70` on every pull request

3. Service layer by file

File	Responsibility	Statements	Executed	Missed	Coverage	
`academic/services.py`	Courses, sections, enrollment, timetable, CSV, assignment	168	151	17	**89.9%**	
`analytics/services.py`	Percentages, shortage, reports	55	55	0	**100.0%**	
`attendance/services.py`	Sessions, marking, edit window, audit	179	172	7	**96.1%**	
`corrections/notifications.py`	Faculty email notification	29	29	0	**100.0%**	
`corrections/services.py`	Submit, queue, approve, reject	81	77	4	**95.1%**	
`users/security.py`	Login lockout (SEC-002)	23	21	2	**91.3%**	
`users/services.py`	Registration, login, logout, password reset	42	37	5	**88.1%**	
Service layer total		**577**	**542**	**35**	**93.9%**	

4. All application code by app

App	Statements	Executed	Coverage	
`users`	257	240	**93.4%**	
`academic`	404	386	**95.5%**	
`attendance`	387	377	**97.4%**	
`corrections`	187	183	**97.9%**	
`analytics`	216	214	**99.1%**	
Total	**1451**	**1400**	96.5%	

5. Statements not executed (service layer)

Most of the 35 unexecuted statements are `except IntegrityError` handlers. They only run when two requests race to insert the same row and the database constraint fires after the service's own duplicate check has passed. The tests cover the duplicate check and the constraint separately, but not the race. The rest are defensive guards for inputs the API layer already rejects. None are in the main paths that the test-plan cases exercise.

File	Line(s)	Function	Code
academic/services.py	46–47	create_course	except IntegrityError as error:
academic/services.py	56	update_course	course.code = code
academic/services.py	64–65	update_course	except IntegrityError as error:
academic/services.py	116–117	create_section	except IntegrityError as error:
academic/services.py	129–130	update_section	except IntegrityError as error:
academic/services.py	146–147	enroll_student	except IntegrityError as error:
academic/services.py	162	add_timetable_slot	raise ValueError("end_time must be after start_time")
academic/services.py	180–181	add_timetable_slot	except IntegrityError as error:
academic/services.py	202	sections_for_user	return queryset.none()
academic/services.py	219	_read_csv_emails	raise CsvEnrollmentError([{"row": None, "error": "File is larger than 1 MB."}])
academic/services.py	240	_read_csv_emails	raise CsvEnrollmentError(
attendance/services.py	164–167	submit_records	except IntegrityError as error:
attendance/services.py	203	sessions_for_user	return queryset.none()
attendance/services.py	249–250	start_session	except IntegrityError as error:
corrections/services.py	63	submit_correction	except IntegrityError as error:
corrections/services.py	65–67	submit_correction	if pending.exists():
users/security.py	29–30	record_failed_login	attempt.failures = 0
users/services.py	42	reset_password	except (
users/services.py	49	reset_password	return False
users/services.py	63	valid_password_reset_token	except (
users/services.py	70	valid_password_reset_token	return False
users/services.py	77	register_user	raise ValueError("Public registration cannot create administrators")

6. Conclusion

NFR-006 is met with a wide margin. The CI pipeline enforces the 70% threshold on every pull request, so coverage cannot drop below the target without a failing build. The line-by-line HTML report

(`htmlcov/index.html` in the evidence archive) shows every file with executed and missed lines highlighted.